

Class members (Fields, Methods and Properties)

Class members



Fields

المتغيرات بتتقسم عندي الى:

Fields -> Normal , static , constants

المتغير العادي اقدر اغير قيمته عادي

ال constants مقدرش اغير قيمتها وتتعرف مره واحده ويتشاف من كل ال objects و ب access عليها عن طريق اسم ال class بس

ال static زي ال constant بيتعرف مره واحده ويتشاف من كل ال objects و ب access عليها عن طريق اسم ال class بس الفرق اني اقدر اغير في قيمته

ال constants و ال statics بيتخزنوا في منطقه اسمها high frequency heap

Methods

الميثودز او الفانكشنز:

بردو عندي الميثود ليها انواع

Methods -> Normal , static

Static Methods

ال static بتتعرف مره واحده وبتتشاف من كل ال objects و ب access عليها عن طريق اسم ال class و لو هستخدم ميثود ثانيه جواها لازم الميثود دي تكون static هيا كمان او اني الميثود دي تكون local جواها

1. "بتتعرف مرة واحدة وبتتشاف من كل ال objects و ب access عليها باسم ال Class"
ال static method تنتمي للكلاس نفسه (Class-level) وليس للنسخ المأخوذة منه (Object-level).
يتم تحميلها في ال Memory (تحديداً في منطقة ال High-Frequency Heap) مرة واحدة فقط عند تشغيل البرنامج، وتكون مشتركة بين الجميع، ويتم استدعاؤها مباشرة هكذا: `ClassName.MethodName()`.
2. "لو هستخدم ميثود ثانية جواها لازم الميثود دي تكون static هي كمان"
الميثود ال Static يمكن استدعاؤها في أي وقت بدون وجود أي Object في الذاكرة. فلو سمحنا لها باستدعاء ميثود عادية (Non-static Instance /)، فالكومبايلر سيتساءل: "هذه الميثود العادية تخص أي Object من ال Objects؟" وبما أنه لا يوجد Object، فالعملية مستحيلة.
لذلك يجب أن تكون الميثود الأخرى static هي الأخرى لتكون جاهزة ومتاحة في الذاكرة دائماً بشكل مستقل.
3. "أو إن الميثود دي تكون local جواها"
ال Local method المكتوبة داخل دالة static يمكن استدعاؤها بحرية كاملة، لأن نطاقها (Scope) محبوس ومضمون بالكامل داخل الدالة ال static نفسها، وبالتالي الكومبايلر لا يحتاج لـ Object لكي يقوم بتشغيلها.

ازاي بكتب الميثود:

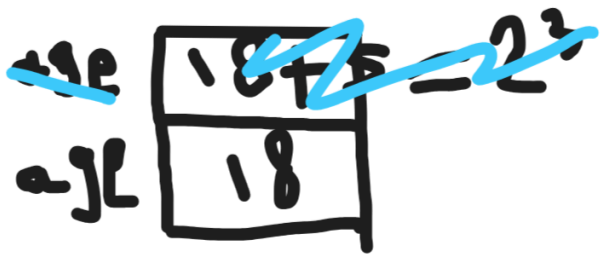
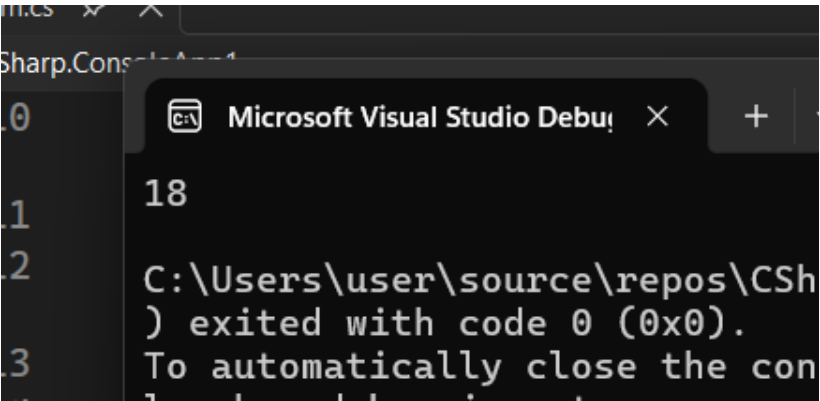
```
<AccessModifier> <datatype>void MethodName (# of parameters)
{
    # series of statements;
}
```

ازاب ب pass ال parameters اول طريقه pass by value

هنا انا لما ابعت متغير و اعمل عليه عمليه معينه في الميثود العمليه عمرها م هتأثر ع المتغير الفعلي لان هنا هما يعتبر اتنين مختلفين اصلا وكل واحد متخزن في مكان مختلف ف الميموري

Pass-by-value is a parameter-passing mechanism where a function receives a **duplicate copy** of an argument's data, ensuring that modifications made inside the function **never alter the original variable**.

```
static void Main(string[] args)
{
    int age = 18;
    fun(age);
    Console.WriteLine(age);
}
public static int fun(int age)
{
    age += 5;
    return age;
}
```



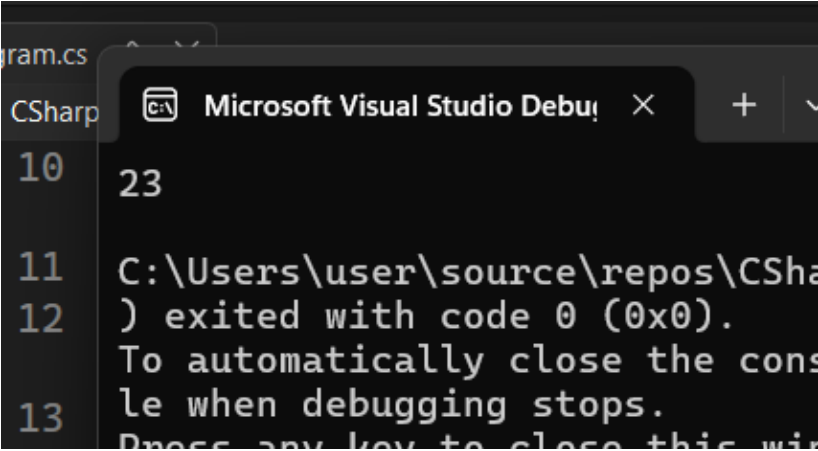
طب انا لو عايز التعديل يغير في القيمة فعليا

هنا انا عايز افهم البرنامج ان هما الاتنين متغير واحد لما تغير في قيمة المتغير جوا الميثود لازم يسمع بره الميثود كمان هنا بق ال **Pass by reference** "ref and out keyword"

بمعنى صحيح الدالة تستقبل Alias لنفس المكان في الميموري

عن طريق استخدام **ref keyword** في ال call و ال definition

```
static void Main(string[] args)
{
    int age = 18;
    fun(ref age);
    Console.WriteLine(age);
}
public static int fun(ref int age)
{
    age += 5;
    return age;
}
```



age $18 + 5 = 23$

Notes :

بستخدم ref لازم المتغير اللي بيعته يكون ليه initial value

طب لو المتغير بتاعي مش هيكون ليه initial value هستخدم **out keyword**

بس برضو خلي بالك لو هعمل عملية حسابيه ع الرقم دا "هزود او اطرح منها" لازم هتخط initial value جوا الميثود او مش بالضبط initial value عادي ممكن انا هعمل عملية حسابيه ومنها هجيب قيمة المتغير الاوليه وبعد كد بق هزود او اطرح منها كد زي منا عايز "لا يمكنك قراءة المتغير (أو عمل عملية حسابية عليه) داخل دالة **out** إلا بعد أن تعينه أولاً داخل الدالة."

تطبيق ع ال **out**

```
var num = "1234";
int number;
if(int.TryParse(num, out number))
{
    Console.WriteLine("valid number "+ number);
}
// حاليا في التحديثات الجديده اقدر اعمل كد
int.TryParse(num, out int number); // بعرف المتغير جوا الداله كمان
```

ملخص لطريقتين ال **pass**

Feature ⓘ	Pass-by-Value	Pass-by-Reference
Data Integrity	Completely safe; protects original data.	Unsafe; vulnerable to unintended changes.
Memory Consumption	Creates a new variable copy in memory.	Shares the existing memory address.
Performance Overhead	Can slow down if copying massive datasets.	Highly efficient even for enormous files.
Scope Effects	Local changes vanish when function ends.	Changes persist globally outside the function.

Method signature “بصمة الدالة”

Name + parameters type + parameters numbers + parameters orders

ودا مهم في ال method overloading

Expression bodies method

بستخدمها لما تكون الميثود بتاعتي بترجع one statement ومفيش اي body تاني
متى يمكنك استخدامها؟
يمكنك تحويل أي ميثود إلى Expression-bodied method بشرط أن تنفذ حاجة واحدة فقط في سطر واحد، سواء كانت ترجع قيمة أو لا ترجع شيء (void)
الداله بتكون كد

```
public string GetWelcomeMessage(string name)
{
    return "Welcome back, " + name;
}
```

باستخدام ال Expression Body

```
public string GetWelcomeMessage(string name) => "Welcome back, " + name;

public void DisplayTime() => Console.WriteLine(DateTime.Now);
```

Local methods

دي ميثود بستخدمها في مكان واحد بس “ميثود واحده بس” بدل م اعرفها في ال class واعملها اكسس موديفير بروح اعرفها جوا الميثود اللي انا بستخدمها فيها بس
لما بعرف الداله دي كمان جوا الداله الاساسيه اللي بتستخدمها اقدر استخدم في الداله بتاعتي متغيرات الداله الاب عطول من غير م امررها ك parameters

Properties

المشكلة اللي عندي

تخيل اني عندي ال problem دي

```
internal class Program
{
    static void Main(string[] args)
    {
        Dollar dollar = new Dollar();
        dollar.Amount = 7.66m;
        Console.WriteLine(dollar.Amount);
    }
}

public class Dollar
{
    public decimal Amount;
}
```

1. مينفعش اي حد ياكسس على ال Amount و يغيرها او يحط فيها اي قيمه طب الحل عادي هعمل constructor و اخلي ال field بتاعي private

```
static void Main(string[] args)
{
    Dollar dollar = new Dollar(7.66m);
    Console.WriteLine(dollar._amount); //طبعا مينفعش

}
}
public class Dollar
{
    private decimal _amount;
    public Dollar(decimal amount)
    {
        if (amount <= 0)
        {
            _amount = 0;
        }
        else
            _amount = amount;
    }

}
```

بس دلوقتي مين هيقرأ أو يغير قيمة amount ؟ مش نافعه
خلاص اخليه public readonly برdo مش نافعه ليه لان انا بالفعل هحتاج في حالات معينه اعدل علي ال amount بره ال constructor
خلاص عندي حل كويس: هعمل Method تاخذ القيمه و Method تانيه تعرضها طبعا دي حاجه مشهوره اللي هي ال getter و setter
نستخدم Methods:

```
static void Main(string[] args)
{
    Dollar dollar = new Dollar(7.66m);
    dollar.SetAmount(9.3m);
    Console.WriteLine(dollar.GetAmount());

}
}
public class Dollar
{
    private decimal _amount;
    public Dollar(decimal amount)
    {
        if (amount <= 0)
        {
            _amount = 0;
        }
        else
            _amount = amount;
    }
    public void SetAmount(decimal value)
    {
        if (value > 0)
            _amount = value;
    }

    public decimal GetAmount()
```

```
{
    return _amount;
}

}
```

شغال تمام، بس بكتب اكواد طويله

بس من هنا جت ال Properties

وخل بالك اصلا CLR بيحول ال property ل `getter` و `setter`

ال Property تسمح لنا نتحكم في الوصول لل `fields` لكن بشكل سهل وكاني بتعامل مع `fields` عادي

بدل:

```
dollar.SetAmount(9.3m);
```

نكتب:

```
dollar.Amount = 9.3m;
```

فالكود هيبق كد

```
static void Main(string[] args)
{
    Dollar dollar = new Dollar(7.66m);
    dollar.Amount = 7.66m; //set
    Console.WriteLine(dollar.Amount); //get
}

}

public class Dollar
{
    private decimal _amount; //Backing Field
    //Property شكل
    public decimal Amount
    {
        get
        {
            return _amount;
        }
        set
        {
            _amount = value;
        }
    }
    public Dollar(decimal amount)
    {
        if (amount <= 0)
        {
            _amount = 0;
        }
        else
            _amount = amount;
    }
}

}
```


ودا ابسط شكل لل Property و بيتكرر كثير ف C# بتوفر اختصار "Auto Property"

Auto Property

```
public class Dollar
{
    public decimal Amount { get; set; }
}
```

أشهر صورة لل Properties

الكومبايلر ينشئ Field مخفيًا تلقائيًا

طبعا اقدر اعمل Validation

هنا تظهر قوة ال Properties

```
public class Dollar
{
    private decimal _amount;
    public decimal Amount
    {
        get
        {
            return _amount;
        }
        set
        {
            if (value <= 0)
            {
                _amount = 0;
            }
            else
            {
                _amount = value;
            }
        }
    }
    public Dollar(decimal amount)
    {
        if (amount <= 0)
        {
            _amount = 0;
        }
        else
        {
            _amount = amount;
        }
    }
}
```

بس خلي بالك هنا انا عندي مشكله ان نفس الكود بيتكرر خلل في مبدأ Don't Repeat Yourself (DRY)

ماشي هحلها اني اعمل Method مسؤوله عن ال validation

```
public class Dollar
{
    private decimal _amount;
    public decimal Amount
    {
        get
        {
            return _amount;
        }
    }
}
```

```
    }
    set
    {
        _amount = ProcessValue(value);
    }
}
public Dollar(decimal amount)
{
    _amount = ProcessValue(amount);
}

private decimal ProcessValue(decimal value) => (value <= 0) ? 0 : value;
}
```

Read Only Property

لو عايزه read فقط:

```
public decimal Amount
{
    get
    {
        return _amount;
    }
}
```

أو:

```
public decimal Amount { get; }
public decimal Amount { get; } = 1.99m //only initial value
```

القيمة تتحدد من ال Constructor

و اقدر اعمالها بشكل **Expression bodies method**

```
public bool IsZero => _amount == 0;
```